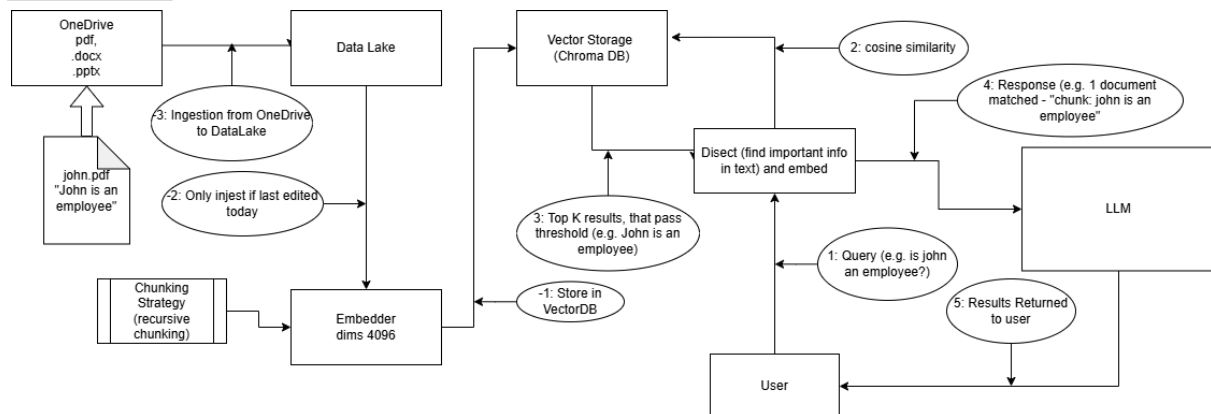


Part A - System Design

Assumptions:

- 2000 Documents that are updated monthly stored in OneDrive which is ported into a DataLake to hold relevant documents
- Documents stored as .pdf, .docx, and .pptx formats
- No cloud or internet access at server layer thus the need for open source models to be run on premise

Architecture:



Ingestion: A nightly cron job polls the document store, parses files using LlamaIndex's native format readers, performs recursive chunking and embedding via Qwen3-Embedding-8, then upserts vectors and metadata into ChromaDB.

Query Time: The user's question is embedded with the same model, ChromaDB returns the chunks that have the top-k cosine similarity scores and pass the similarity threshold, these are injected into a prompt template sent to the vLLM server. The response streams back through the API gateway.

Key Decisions & Tradeoffs

Embedding Model: The embedding model used will be "Qwen3-Embedding-8B" this model was chosen due to its high performance on text embedding and retrieval tasks, while also having multilingual capabilities which may be important given our Singaporean context as some documents may be in languages other than English. Models such as "F2LLM-v2-14B" and "llama-embed-nemotron-8b" were considered as well but the earlier was not much larger in parameters while having worse performance while the later had similar performance but was edged out by our chosen model in the MTEB leaderboards.

Inference Engine: As for the inference engine used, I will use vLLM running an AWQ-quantized version of "Qwen3-235B-A22B". vLLM was chosen over a standard Hugging Face pipeline because its Continuous Batching and PagedAttention mechanisms dynamically manage GPU memory, allowing the system to handle concurrent users without VRAM fragmentation or OOM crashes. "Qwen3-235B-A22B" was selected over alternatives like "MiniMax-M2.7" due to its explicit focus on multilingual coverage, while "MiniMax-M2.7" is primarily optimised for coding workflows.

Monitoring

Retrieval quality: log cosine similarity scores per query; a drop signals embedding drift or document format changes

Latency: track time-to-first-token and total generation time via vLLM's /metrics endpoint Ingestion health – alert if the nightly worker fails or document count deviates unexpectedly

Full Question Response time: track total time taken to complete response, from receiving the query, to completing output.

Production Failure Mode

Contradictory chunk retrieval: In testing, queries hit clean single-topic chunks. In production, an updated organisation policy spread across two documents ingested months apart returns both chunks at similar similarity scores. This could happen due to people uploading documents without removing the older version. The LLM then generates an answer that is factually wrong. This would not surface in evaluation because test sets use self-consistent documents.

Part C - Opinion

Firstly, I would log every production query with its retrieved chunks, similarity scores, assembled prompt, and final response, then manually trace user-reported failures.

Next, I would compute a 1 week average of the cosine similarity score of the top K chunks returned across the past 6 months to identify a change in similarity scores. Assuming the documents are correctly updated and the embedding model is unchanged, a declining score indicates query drift. Next, we compare the similarity score between each week's queries, against the queries within the first week. If this is decreasing, then there may be changes in terminology, types of questions, or an increase in query complexity.

Finally, I would check the retrieval rate of recently updated documents by looking at the frequency of top-k documents containing updated documents. If updated documents are not being retrieved, I would check ChromaDB's metadata to find chunks which map to a document with a newer than indicated last_modified timestamp. If found, it means old chunks were not evicted and are competing with updated ones at retrieval time, causing outdated answers.